

Project 2 report: Machine learning-based predictions of rod temperature and pressure in a nuclear reactor

Goktug Ilter, Guillaume Salha and Jean Lefort

Abstract—The accurate prediction of the temperature and pressure of nuclear reactor rods is imperative for the efficacy of safety simulations. While traditional physics-based simulations offer precise results, they are computationally intensive. This project investigates the application of machine learning (ML) algorithms for the fast prediction of these values, allowing for more complex nuclear reactor simulation. Using the open-source software OFFBEAT (V9), 500 time-dependent simulations were generated, forming the basis of the 100'000 point dataset. A benchmark was conducted on several ML models to evaluate their predictive capabilities and computational efficiencies. The study demonstrates that advanced models such as random forests and XGBoost exhibit superior performance in terms of accuracy, with root mean square errors (RMSE) as low as 2 K and 0.05 bar for temperature and pressure predictions. The analysis also highlights the effectiveness of feature expansion, which enables simple models such as linear regression to achieve results comparable to those of complex models. Finally, the incorporation of gaussian noise and outliers further underline the resilience of the prediction methods against perturbations, thereby validating their practical utility.

I. INTRODUCTION

Accurate prediction of the temperature and pressure within the rods of a nuclear reactor core is imperative in reactor simulations, as these parameters influence its physical behavior (e.g. mechanical cracks or neutron cross-section).

The objective of this project is to develop surrogate machine learning (ML) models that leverage simulation data from OFFBEAT ([1]). These models predict rod temperature (T_{rod}) and pressure (P_{rod}) based on key physical and geometric parameters. The employment of these ML models facilitates the execution of inference processes at a rate that is significantly superior to that of conventional OFFBEAT methods, thereby enabling the execution of more complex analyses.

The use of ML models in this field has previously been employed for predicting peak cladding temperatures ([2]). In this study, we are building upon this approach by implementing and benchmarking multiple ML methods. The parameters investigated for their influence on T_{rod} and P_{rod} are the following: linear heat generation rate (`lhgr`), the fuel radius of the rod (`fuel_radius`), thickness of the metal layer that surrounds the nuclear fuel pellets (`clad_thickness`), coolant temperature (`coolant_temperature`), gap between the fuel rod and the zirconium alloy cladding (`gap_size`), and time (`time`).

II. MODELS AND METHODS

A. Data Cleaning and Engineering

a) Dataset Creation: We have created the dataset by aggregating results from 500 time-dependent simulations, each producing approximately 200 data points. This resulted in

a dataset with around 100,000 rows, which we considered sufficient given the relatively small number of features. Each row includes the six features (`lhgr`, `fuel_radius`, `clad_thickness`, `coolant_temperature`, `gap_size`, `time`) and two outputs: `rodPressure` and `volAverage(T)`. The temperature is expressed in Kelvin (K), while pressure, initially in Pascal (Pa), is converted to Bar for readability. Data was split into training and testing sets using stratified binning to ensure balanced coverage across the output space, a useful strategy for skewed regression targets.

To maximize the coverage of the parameter space, we employed Latin Hypercube Sampling (LHS) instead of simple uniform sampling for each parameter, ensuring a more efficient exploration of feature combinations given our freedom to create our dataset.

b) Elimination of Initial Time Steps: The first two time steps ($t = 0$ s and $t = 1$ s) exhibited sharp and abrupt changes in rod temperature (T_{rod}), transitioning from 300K to higher values. These subtransient states are challenging to model and not relevant to our focus on steady-state behavior. Following discussions with the lab, we excluded these time steps to improve model accuracy and focus on steady-state predictions for an ultimate goal as a real-time anomaly detection.

c) Outliers and Standardization: The dataset, generated from controlled simulations, showed no observable NaNs or outliers in the data plots, indicating no significant issues such as non-convergence. All feature columns were standardized prior to training to ensure consistent scaling for the models (though, for models like random forest regressor, standardization has an impact of about $\Delta_{RMSE} = 10^{-2}K$).

B. Machine Learning Models

We implemented and benchmarked several machine learning models of increasing complexity to explore both linear and nonlinear relationships between features and outputs.

a) Evaluation Metrics: Since the outputs (T_{rod} and P_{rod}) are continuous, we used regression-specific metrics to evaluate model performance:

- **Root Mean Square Error (RMSE):** Highlights sensitivity to large errors.
- **Mean Absolute Error (MAE):** Less sensitive to outliers, providing a robust error estimate.

- **R^2 Score:** Measures the proportion of variance explained by the model.

These metrics allowed for a comprehensive assessment of model accuracy and robustness.

Model	Parameters
Least Squares Linear Regression	(no hyperparameters)
Ridge Regression	Rod Pressure: alpha = 10.0, Temperature: alpha = 0.1
Support Vector Regressor (SVR)	C = 10.0, kernel = 'rbf'
K-Nearest Neighbors	n_neighbors = 3, weights = 'distance'
Random Forest Regressor	n_estimators = 50
LightGBM	learning_rate = 0.1, n_estimators = 200, num_leaves = 40
XGBoost	learning_rate = 0.1, max_depth = 7, n_estimators = 200
Fully connected neural network	L=3 layers K=15 neurons

TABLE I: Implemented Models and Key Parameters

b) Implemented Models and Parameters: All models were evaluated using 5-fold cross-validation to ensure fair benchmarking and robustness across datasets. Validation errors were computed. 10% of the dataset was kept away to eventually test the models, and provide test errors.

Different hyperparameters were tuned separately for rod pressure and temperature models using grid search, optimizing mean validation scores from cross-validation.

c) Hyperparameter Tuning: For models with tunable hyperparameters (e.g., Random Forest, LightGBM, XGBoost), we optimized parameters using a grid search approach, selecting the configuration with the highest cross-validated performance. Specific parameter tuning details are omitted here, and detailed in the results section.

III. RESULTS

A. Comparison of models' performances

We assessed the performances of all our models on the dataset, and we gathered the results in tables II and III. Results from hyperparameter tuning do not appear in this table, since for most models, a pre-tuning was performed and only the optimal choice of parameters was kept for the values in these tables. For some models, this tuning will be explored in more details in the coming sections. Each table displays both the **validation errors (in black, with mean $\pm$ standard deviation)** and the **test errors (in blue, only one test error)**.

B. Hyperparameters tuning

Most Machine learning models can be parametrized. Hyperparameters (e.g. number of trees in the random forests) can be chosen to complexify or simplify the model. If models are too simple (low N_{trees}), the complexity of the data may not be

	RMSE (K)	MAE (K)	R^2
Least squares linear regression	38.70 $\pm$ 0.39 38.60	27.69 $\pm$ 0.19 27.60	0.9653 $\pm$ 0.0005 0.9655
Ridge regression	38.79 $\pm$ 0.39 38.60	27.69 $\pm$ 0.19 27.60	0.9653 $\pm$ 0.0005 0.9655
Support vector regressor	13.67 $\pm$ 0.69 14.72	11.34 $\pm$ 0.08 11.25	0.9949 $\pm$ 0.0005 0.9950
K nearest neighbours	14.59 $\pm$ 1.38 15.56	4.73 $\pm$ 0.24 4.72	0.9949 $\pm$ 0.0009 0.9944
Random forests	2.32 $\pm$ 0.22 2.19	0.594 $\pm$ 0.013 0.590	0.99988 $\pm$ 0.00003 0.99989
LightGBM	3.49 $\pm$ 0.33 3.18	2.11 $\pm$ 0.04 2.14	0.9997 $\pm$ 0.0001 0.9998
XGBoost	2.43 $\pm$ 0.05 2.39	1.44 $\pm$ 0.02 1.41	0.9999 $\pm$ 0.0000 0.9999
Fully connected neural network	14.79 $\pm$ 1.19 13.06	8.82 $\pm$ 0.67 8.28	0.9949 $\pm$ 0.0008 0.9960

TABLE II: **Validation** and **test** errors of different models predicting **rod temperature**.

	RMSE (bar)	MAE (bar)	R^2
Least squares linear regression	3.8338 $\pm$ 0.03347 3.7850	2.7258 $\pm$ 0.0163 2.7082	0.7912 $\pm$ 0.0018 0.7940
Ridge regression	3.8338 $\pm$ 0.03347 3.7850	2.7258 $\pm$ 0.01638 2.7080	0.7912 $\pm$ 0.0018 0.7940
Support vector regressor	0.6244 $\pm$ 0.0066 0.6372	0.5006 $\pm$ 0.0046 0.5028	0.9945 $\pm$ 0.0002 0.9942
K nearest neighbours	0.4274 $\pm$ 0.0025 0.4224	0.2419 $\pm$ 0.0095 0.2440	0.9974 $\pm$ 0.0003 0.9974
Random forests	0.0477 $\pm$ 0.0017 0.0471	0.0327 $\pm$ 0.0007 0.0327	0.99997 $\pm$ 0.00000 0.99997
LightGBM	0.2389 $\pm$ 0.0019 0.2320	0.1777 $\pm$ 0.0016 0.1742	0.9992 $\pm$ 0.0000 0.9992
XGBoost	0.1674 $\pm$ 0.0015 0.1644	0.1214 $\pm$ 0.0007 0.1202	0.9996 $\pm$ 0.0000 0.9996
Fully connected neural network	2.0803 $\pm$ 0.0315 2.0406	1.3639 $\pm$ 0.0169 1.3317	0.9384 $\pm$ 0.0014 0.9407

TABLE III: **Validation** and **test** errors of different models predicting **rod pressure**.

captured. If N_{trees} increases, the model risks to overfit (capture even the noise in the data). Therefore, representing the total error as a function of the hyperparameter N_{tree} enables to find the minimum error and the optimal N_{tree} (or more broadly, the best hyperparameter) associated. The random forest regressor was run with 10, 20, 50, 100, 150, 200 trees, with 5-fold cross-validation. The means and standard deviations of the RMSE, MAE, and R^2 were computed and represented on the figure 2 (only for **temperature prediction**). They highlight that as soon as we reach 50 trees, RMSE and MAE almost reach their minimum values and R^2 its maximum. However, with increasing numbers of trees, the computation time increases. Therefore, we concluded that selecting 50 trees was optimal, and we kept this hyperparameter for the values that were presented in table II.

The same figure is available in the code for **pressure prediction**, and similar conclusions can be drawn, though the optimal N_{trees} may be closer to 70.

IV. DISCUSSION

A. Robustness of the Models to noise and outliers

We assess the robustness of our models on less clean test datasets, to simulate real-world measurement inaccuracies.

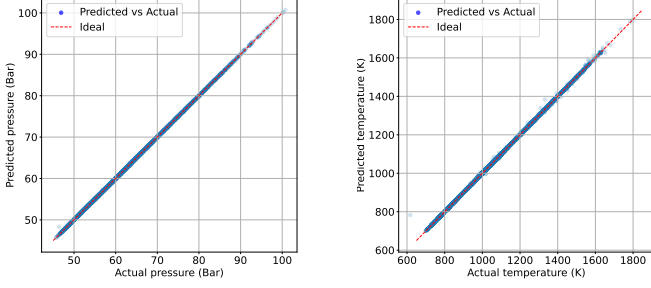


Fig. 1: Comparison of predicted vs actual values for rod Pressure (left) and Temperature (right) using random forests with 50 trees.

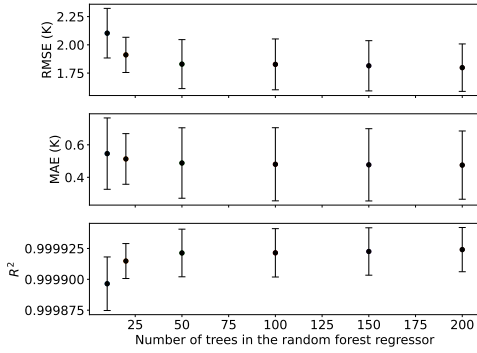


Fig. 2: Errors and R^2 for random forest regressor for temperature prediction, for various N_{trees}

While features such as fuel radius, clad thickness, gap size, and time are typically measured with minimal error, others—namely coolant temperature and linear heat generation rate (LHGR)—may contain noise or inaccuracies.

We analyzed the effect of adding **noise** in coolant temperature and LHGR on the quality of predictions. Figure 3 demonstrates some models' performances when adding noise in LHGR (top plot) and in the output values (bottom plot). Additionally, we trained our models with noisy target data (T_{rod} or P_{rod}) to assess their ability to detect potential anomalies in simulation outputs.

Outliers were artificially introduced in key features to evaluate their impact on model robustness. The top plot of figure 4, reveals that the presence of outliers in LHGR had a significant effect on predictions.

Furthermore, we diluted the time instances to simulate scenarios with fewer temporal measurements, making the modeling task more challenging. Interestingly, this had minimal effect on prediction performance for both rod pressure and temperature, as illustrated in the time dilution graph (Figure 5). This reveals that we could extract only 50% of the time steps of the simulations (which would divide the dataset size by two) and keep excellent results.

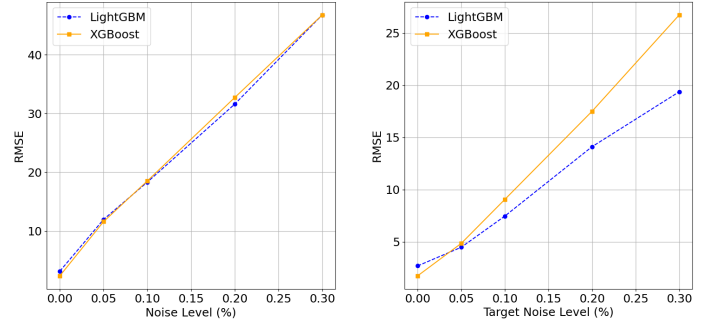


Fig. 3: Impact of noise on lhgr (left) and temperature (right) on the performances.

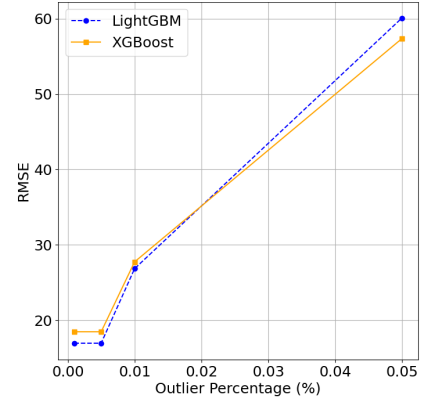


Fig. 4: Impact of outliers in lhgr on the performances

Finally, we performed overfitting analysis to ensure the model's generalizability across various scenarios. Figure 6 highlights the results of this analysis.

Our findings indicate that LHGR is a critical feature, as even small variations significantly impact predictions. In contrast, coolant temperature has a lesser effect on model performance, as shown by the comparative graphs (in the code only).

B. Feature expansion

In this work, we implemented feature expansion using the `PolynomialFeatures` class from the `scikit-learn` library. This technique generates polynomial terms and interaction features up to a degree d , where d is the chosen expansion degree. For example, with two initial variables x_1 and x_2 , a degree $d = 2$ expansion creates new features such as x_1^2 , x_2^2 , and x_1x_2 . This transformation is performed while excluding explicit bias terms, ensuring a consistent structure in the data.

This enables to capture complex nonlinear relationships between the input features and the target variable, which could enhance the performance of simple linear models. This approach may increase the risk of overfitting, particularly for high expansion degrees. Regularization can help preserve the model's generalization capability. However, we have not see any increase of the RMSE in the test set before maximum degree=10, which

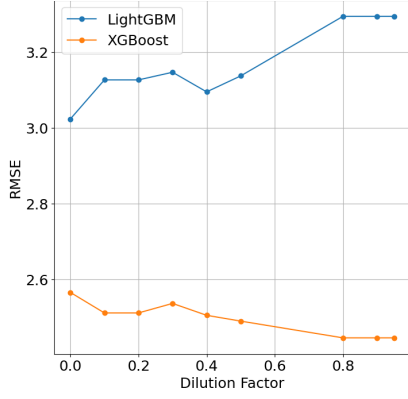


Fig. 5: RMSE vs Dilution factor for time steps

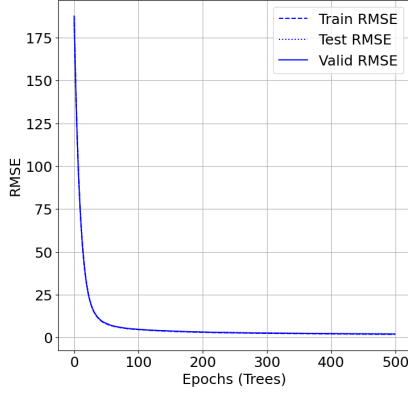


Fig. 6: Overfitting analysis

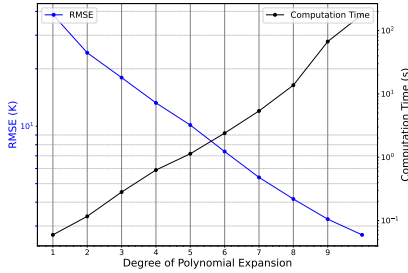


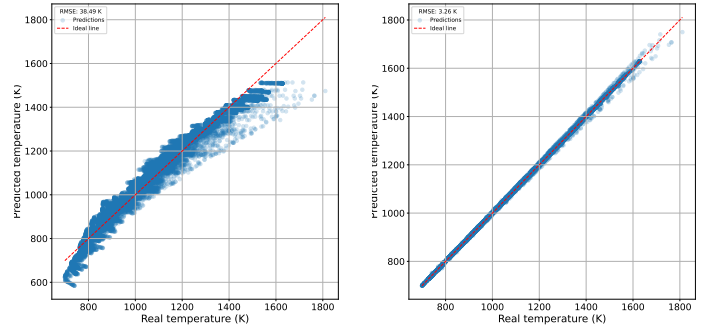
Fig. 7: Trade-off between RMSE and computation time for feature expansion.

is the limit of our mean of computation, as presented in Figure 7.

As shown in Figure 8, this feature expansion with a maximal degree of 9 has considerably reduced the RMSE on the test set, from 38.49 K to 3.26 K.

C. Model Choice

We choose the best model from the tables II and III. Since there are non-linearities, the complex models such as random forest regressor and XGBoost get the best out of the game. Both for pressure and temperature predictions, we end up finding that the random forests with 50 trees perform best, whether we look at RMSE or MAE.



(a) Linear regression without feature expansion.

(b) With polynomial feature expansion, max degree = 9).

Fig. 8: Comparison of Linear Regression with and without feature expansion.

V. CONCLUSION

During this project, the goal was to develop machine learning models to predict rod temperature and pressure. This aims at replacing existing more time consuming physics-based simulations. We built various models, from simple ones (linear regression) to complex ones (such as fully-connected neural network or XGBoost).

We demonstrated the great performances of random forests and XGBoost, both for predictions of temperature and pressure. In particular the random forests regressor seems the most accurate model and leads to an RMSE of about 2K for predicted temperatures around 1000K, and of 0.05 bar for predicted pressures around 100 bar. The MAE are even lower, 0.6K and 0.03 bar.

We have also seen that quite consistently, the MAE was lower than the RMSE (by a factor 1.3-2 in general). This is likely explained by the values at high temperature and pressure, which are a little underpredicted. Indeed, there are fewer of them in the dataset. This might be explained by physical phenomena that are significant only when T is high (not necessarily due to high T but due to other parameters that lead T to be high, such as $lhgr$).

Rich discussions with the lab helped us define some possible next steps for this project. One opportunity would be to integrate the fission gas release in the machine learning model. This phenomenon can be critical at higher burnup values. Also, incorporating the history of the features (the evolution with time, or the integral over time) as new features could prove relevant. This might be complex to implement though. Last, our work could be generalized with various axial power profiles. With these additions, such model would be very useful for real-time anomaly detection system, which would help prevent accidents and ensure safety in nuclear reactor cores.

REFERENCES

- [1] A. Scolaro, I. Clifford, C. Fiorina, and A. Pautz, "The offbeat multi-dimensional fuel behavior solver," *Nuclear Engineering and Design*, vol. 358, p. 110416, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0029549319304479>
- [2] W. Sallehuddin and A. Diab, "Using machine learning to predict the fuel peak cladding temperature for a large break loss of coolant accident," *Frontiers in Energy Research*, vol. 9, p. 755638, 2021.
- [3] EPFL, "How to use the digital ethics canvas?" 2024, accessed: 2024-11-28. [Online]. Available: <https://www.epfl.ch/education/educational-initiatives/cede/training-and-support/digital-ethics/a-visual-tool-for-assessing-ethical-risks/the-digital-ethics-canvas-how-to/>

VI. ETHICAL ASSESSMENT

The methodology followed in this section aligns with the approach outlined in the Digital Ethics Canvas framework, as described by EPFL [3].

A. Welfare

These ML algorithms serve as a predictive tool for temperature and pressure in nuclear fuel rods, aiming to assist nuclear engineers in accelerating accident scenario simulations. While enhancing efficiency, inaccuracies in predictions could compromise safety decisions. Thus, the model should only be applied within its validated input and output ranges, particularly for temperature and LHGR, as these parameters are critical for safety. This tool is intended solely for analytical acceleration; safer, validated models are required for real-world implementations.

B. Fairness

The tool is designed for qualified nuclear safety engineers, limiting accessibility to a specialized group. Biases may arise if the training data does not represent all reactor conditions, potentially skewing predictions. However, it remains accurate within the range of primary inputs. To address these issues, our solution includes explanations for predictions to ensure correct use.

C. Autonomy

Engineers using this prediction tool should understand its limitations (especially in terms of input/output ranges) and the confidence of the model used. This is, for example, indicated by the result of the cross-validation section, which provides an idea of how likely a significant error is for each model. This tool allows engineers to override suggestions, preserving their autonomy in decision-making. Additionally, the algorithms used to produce the models are open-source, enabling users to modify them to include relevant new features as inputs. Transparency in error margins and methodology should ensure that users can make well-informed choices during simulations.

D. Privacy

Our algorithms require numerical data from reactor simulations to be trained but do not involve personal or sensitive information. Access to this data is restricted, as it is only available through non-public simulations of the specific reactor. However, since the tools used to create the database (OFFBEAT from OpenFOAM) are open-source, it might hypothetically be possible to reproduce the data. In this sense, our model is not more dangerous than having access to this simulation software.

E. Sustainability

The solution's carbon footprint is minimal, primarily arising from computational resources for model training and simulation. This impact comes from our computers, but also from the indirect carbon footprint of training AI tools used to assist in debugging codes and reformulating sentences, namely ChatGPT, Perplexity.ai, and DeepL).